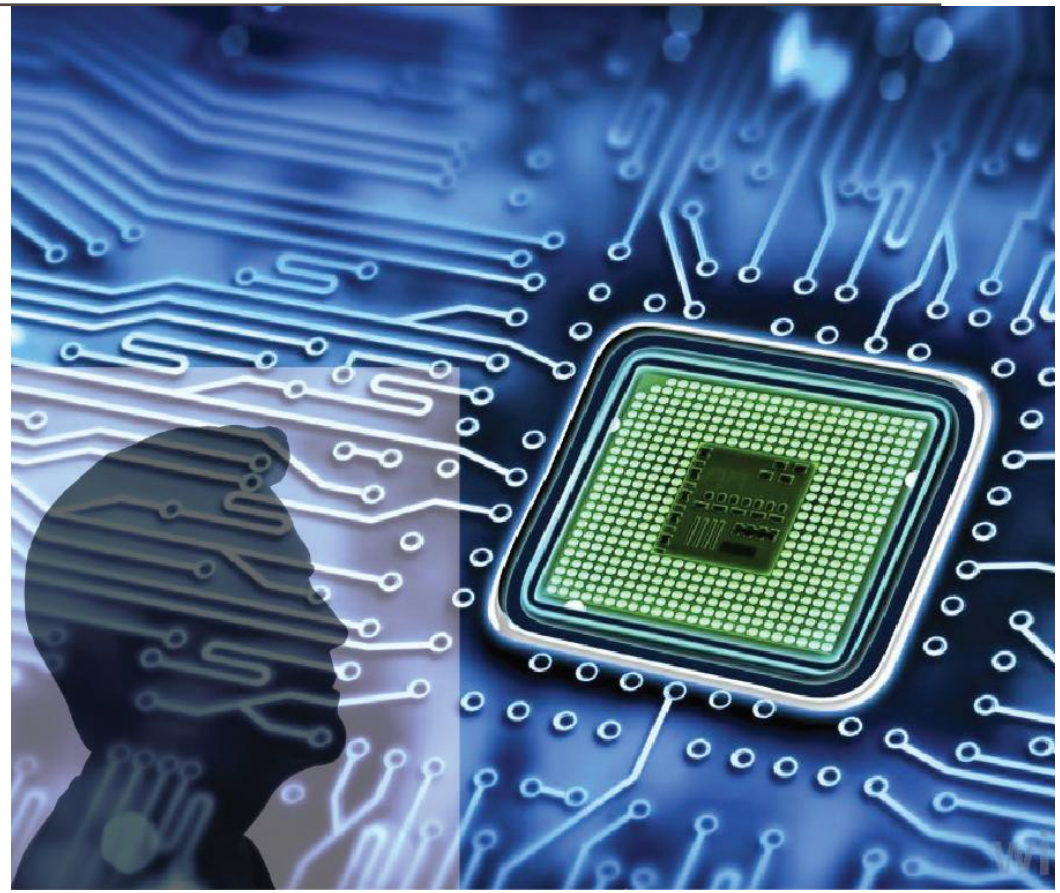
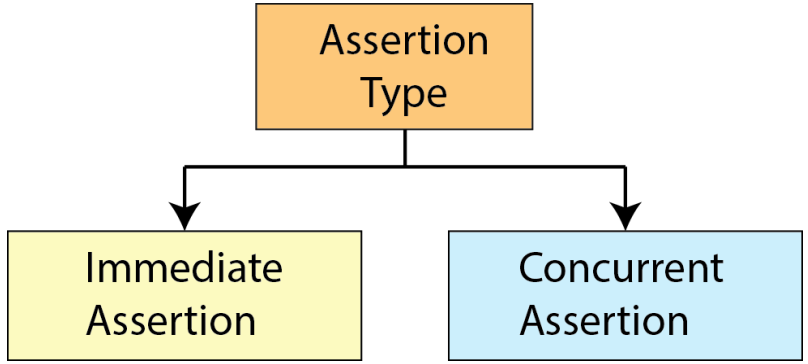
ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-12 Lecture-1
SystemVerilog Assertions



Sneh Saurabh
11th November 2025

Types of Assertions (Recap)



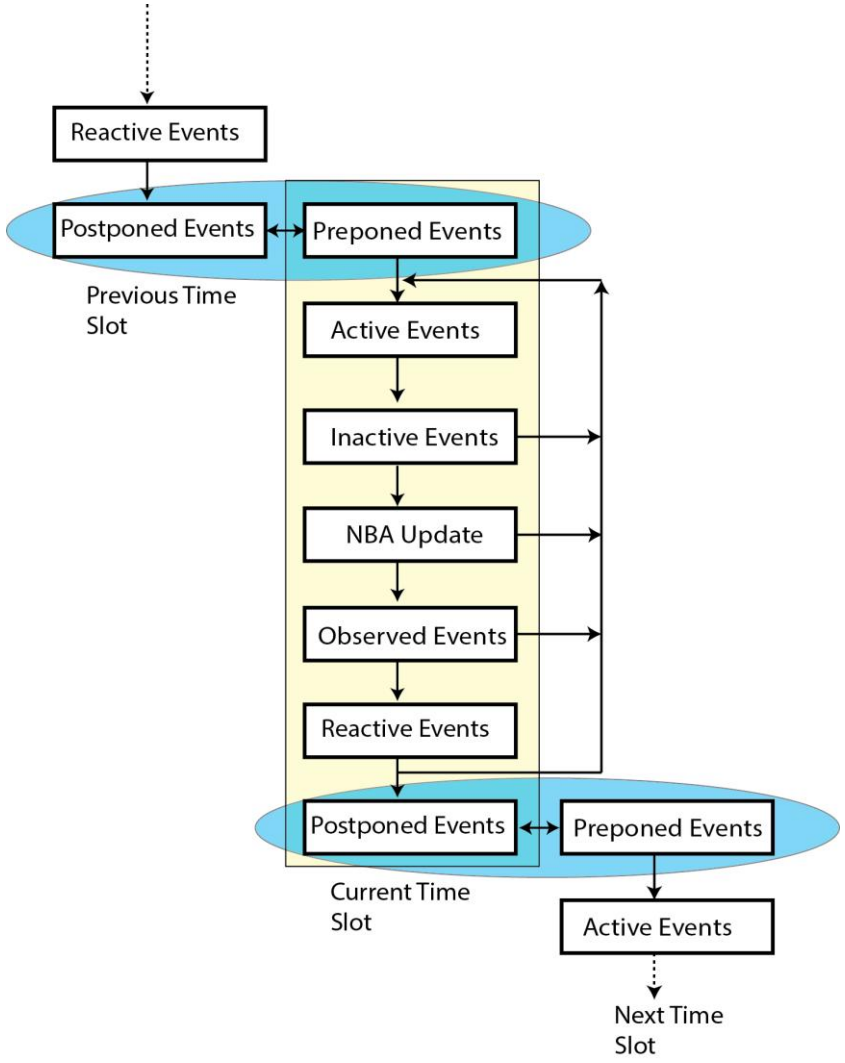
Immediate assertions:

- Simple assertions checked whenever they are visited in the procedural code
- Allow only Boolean arguments
- No mechanism of clocking or reset
- Do not support advance property operators (passage of time)
- Keyword assert without keyword property

Concurrent assertions:

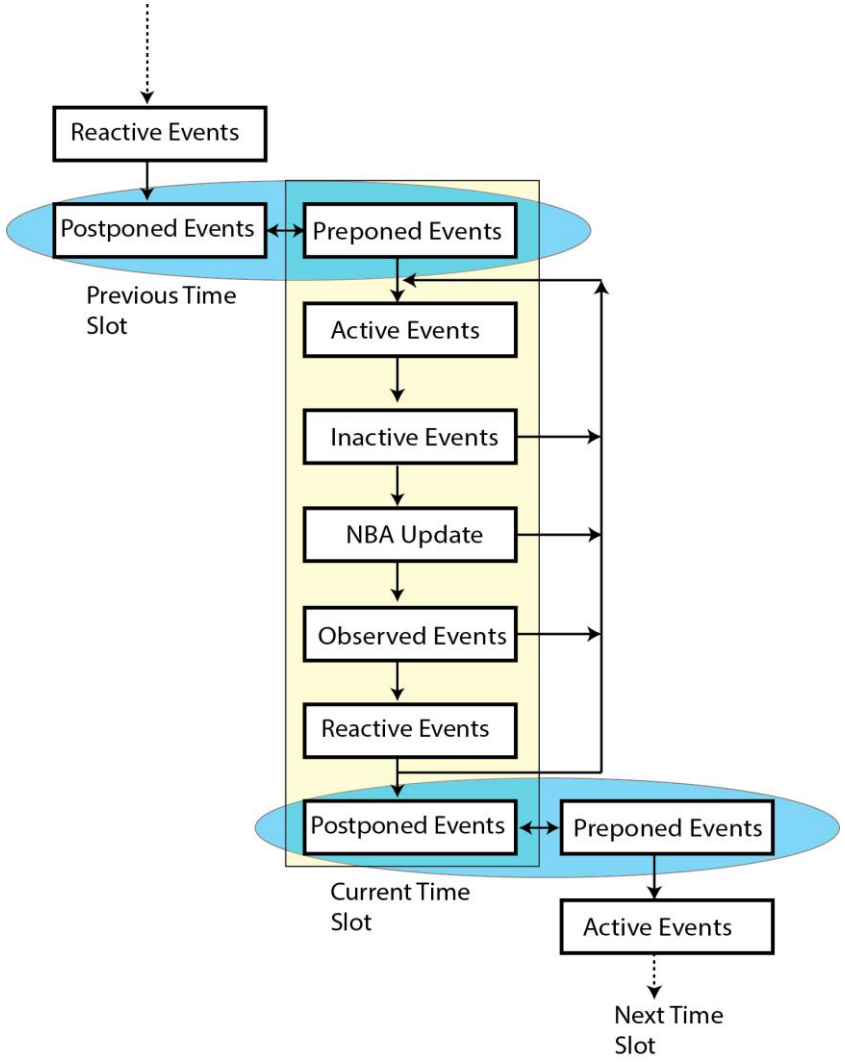
- Always evaluated relative to one or more clocks
- Can describe behavior that spans time
  - Advanced operators such as logical implication over time intervals
- More useful than immediate assertion
- Keyword assert with keyword property

SystemVerilog: Timing Regions (Recap)



1. Design events run (RTL, gate-level and clock generator): **Active, Inactive** and **NBA update**
2. Observed region: SystemVerilog **Concurrent** Assertions evaluated.
3. Reactive region: Testbench code in a program executes.
4. Postponed region: samples signals at the end of the time slot, in the read-only period, after design activity has completed ($monitor, $strobe part of it)

Evaluation of Immediate Assertions



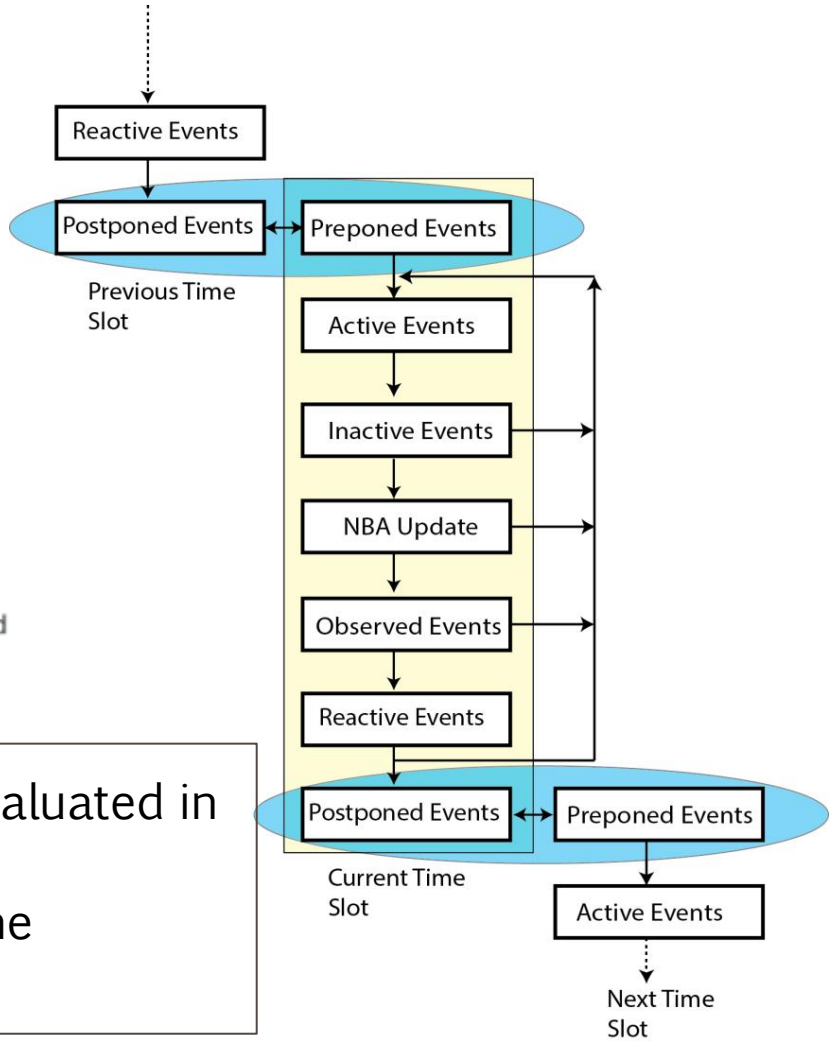
Immediate assertions:

- Immediate Assertions are executed as procedural code and evaluated in the Active Region
- Values of the current variable in the Active region is taken in the evaluation

Example: Simple Immediate Assertion (Simulation)

```
1 module top(input clk, input reset);
2   reg a;
3
4   initial begin
5     a = 0;
6   end
7
8   assign not_a = !a;
9
10  always_comb begin
11    a1: assert (not_a != a) $display("Passed: a=%0d not_a=%0d", a, not_a);
12          else $display("Failed: a=%0d not_a=%0d", a, not_a);
13  end
14 endmodule
```

```
ncsim> run
ncsim: *E,ASRTST (./sva-immediate-assert-1.sv,14): (time 0 FS) Assertion top.a1 has failed
Failed: a=0 not_a=x
Passed: a=0 not_a=1
```



- Immediate Assertions are executed as procedural code and evaluated in the Active Region
- Values of the current variable in the Active region is taken in the evaluation

Example: Simple Immediate Assertion (Formal)

```
1 module top(input clk, input reset);
2   reg a;
3
4   initial begin
5     a = 0;
6   end
7
8   assign not_a = !a;
9
10  always_comb begin
11    a1: assert (not_a != a) $display("Passed: a=%0d not_a=%0d", a, not_a);
12          else $display("Failed: a=%0d not_a=%0d", a, not_a);
13  end
14 endmodule
```

```
0.0.PRE: A proof was found: No trace exists. [0.00 s]
INFO (IPF057): 0.0.PRE: The property "top.a1" was proven in 0.00 s.
0: Found proofs for 1 properties in preprocessing
```

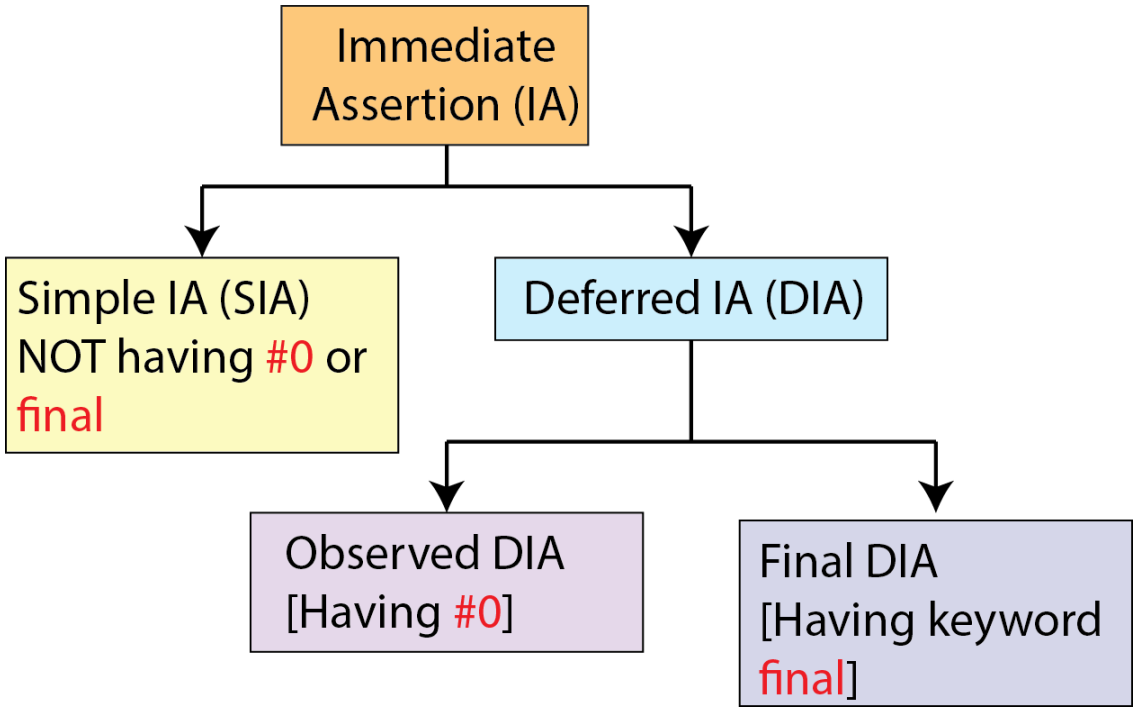
```
Properties Considered              : 1
      assertions                   : 1
        - proven                   : 1 (100%)
        - bounded_proven (user)    : 0 (0%)
        - bounded_proven (auto)    : 0 (0%)
        - marked_proven            : 0 (0%)
        - cex                      : 0 (0%)
        - ar_cex                   : 0 (0%)
        - undetermined             : 0 (0%)
        - unknown                  : 0 (0%)
        - error                    : 0 (0%)
```

Immediate assertions:

- Prone to fail due to glitch
- However, we typically don't want to see the effect of glitches in assertions (but want behavior similar to the formal tool)

How can we avoid failure of immediate assertions for transient (glitch) values?

Types of Immediate Assertions



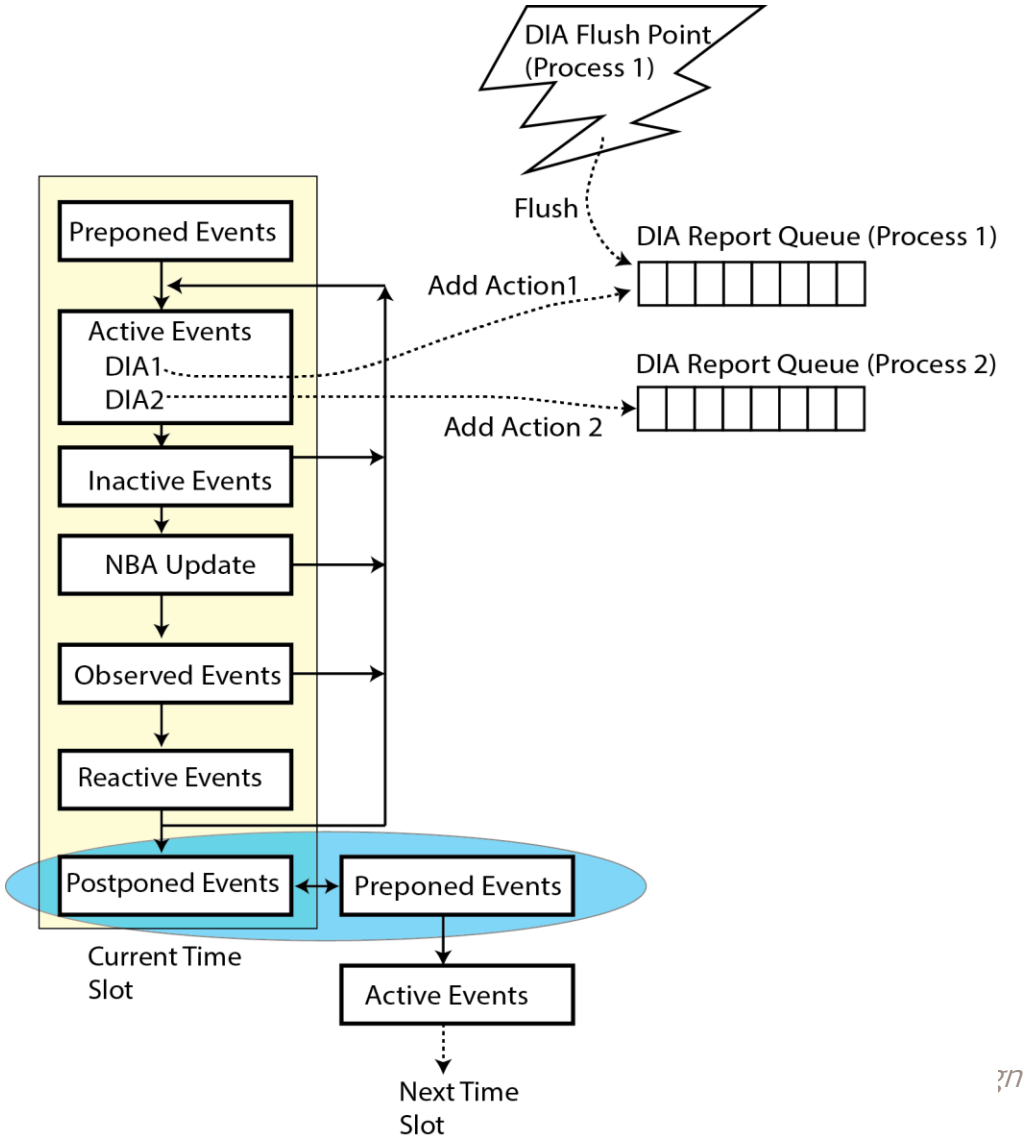
Simple Immediate Assertions (SIA):

- Pass and Fail Actions take place immediately upon assertion evaluation.

Delayed Immediate Assertions (DIA):

- Pass and Fail Actions are delayed until later in the time step
- Provides some level of protection against unintended multiple executions on transient (glitch) values

Mechanism of Reporting for DIA (1)

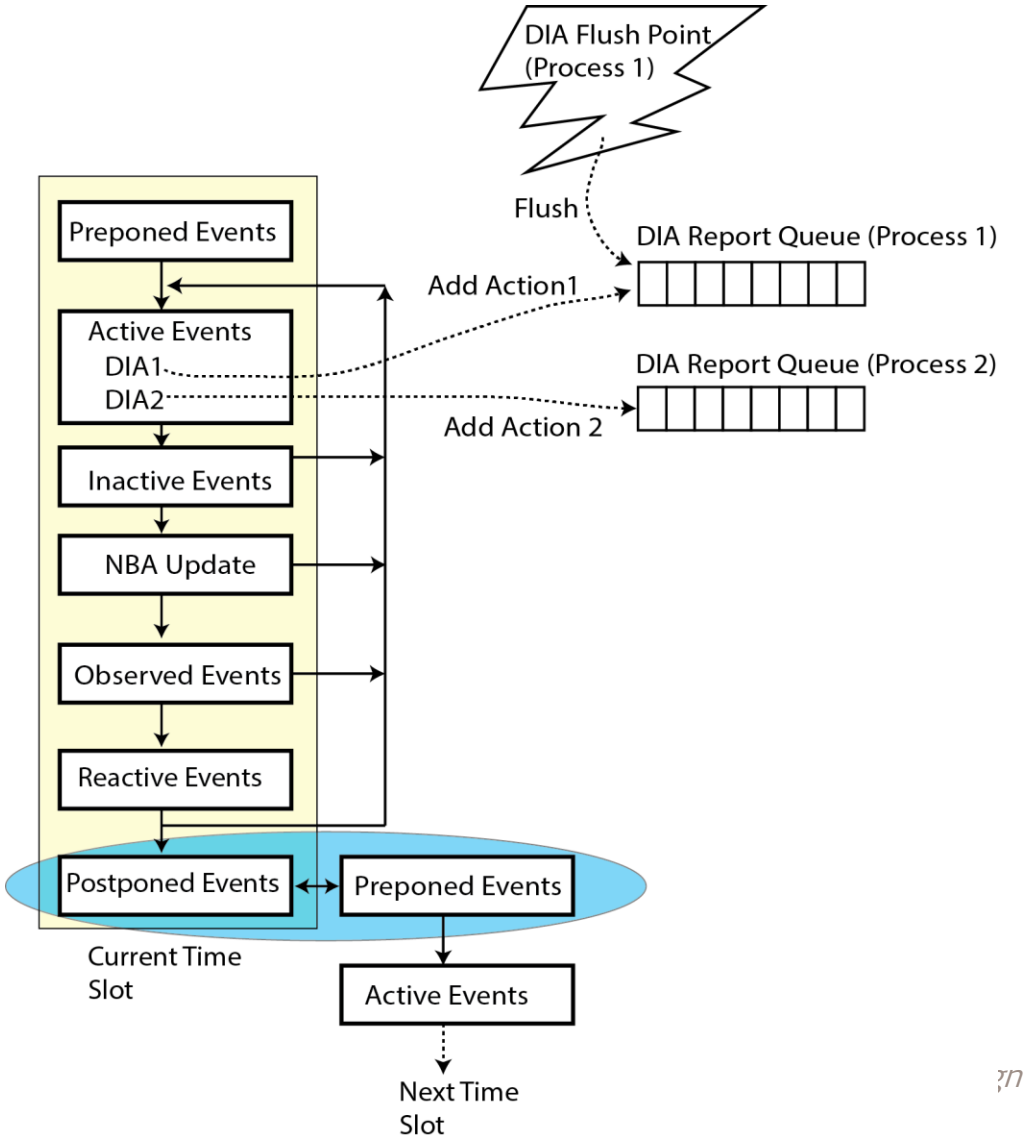


- Pass and Fail Actions are not reported in Active Region
- It is added to DIA Report Queue
  - Every process has a separate DIA Report Queue

- DIA Report Queue is flushed (emptied) at flush points
  - Example: process is declared by an **always_comb** or **always_latch**, and its execution is resumed due to a transition on one of its dependent signals

- Flushing of DIA Report Queues lead to removal of transient pass/fail to be reported

Mechanism of Reporting for DIA (2)



Observed DIA (#0)

- **In the Observed Region:** all existing actions for Observed DIA in the DIA Report Queue matures (or confirmed for reporting)
- Corresponding Actions are executed in the Reactive Region
  - If the code in the Reactive Region modifies signals and causes another pass to the Active Region to occur, this still may create glitching behavior

Final DIA (keyword final)

- **In the Postponed Region:** all pending actions For Final DIA in the DIA Report Queue matures (or confirmed for reporting)
- Corresponding Actions are executed in the Postponed Region (No iteration possible)
- Will not show transient or glitchy behavior

Observed Deferred Immediate Assertions (DIA)

```
1 module top(input clk, input reset);
2   reg a;
3
4   initial begin
5     a = 0;
6   end
7
8   assign not_a = !a;
9
10  always_comb begin
11    a_observed: assert #0 (not_a != a)
12      $display("Passed obs: a=%0d not_a=%0d", a, not_a);
13      else
14      $display("Failed obs: a=%0d not_a=%0d", a, not_a);
15  end
16 endmodule
```

```
ncsim> run
Passed obs: a=0 not_a=1
ncsim: *W,RNQUIE: Simulation is complete.
ncsim> exit
```

- When *a* changes, simulator can evaluate assertion *a_observed* twice:
  1. For the change in *a*
  2. For the change in *not_a* (after the evaluation of the continuous assignment)

- The failure during the first execution of *a_observed* will be added in the DIA Report Queue
- When *not_a* changes, DIA Report Queue is flushed due to the activation of the always_comb block (flush point reached)
- No failure of *a_observed* will be reported.

- Success will be added to DIA report queue
- Mature in the Observed Region
- Reported in the Reactive region

Final Deferred Immediate Assertions

```
1 module top(input clk, input reset);
2   reg a;
3
4   initial begin
5     a = 0;
6   end
7
8   assign not_a = !a;
9
.0  always_comb begin
.1    a_final: assert final (not_a != a)
.2      $display("Final Passed: a=%0d not_a=%0d", a, not_a);
.3      else
.4      $display("Final Failed: a=%0d not_a=%0d", a, not_a);
.5  end
.6 endmodule
```

```
ncsim> run
Final Passed: a=0 not_a=1
ncsim: *W,RNQUIE: Simulation is complete.
ncsim> exit
```

- When *a* changes, simulator can evaluate assertion *a_final* twice:
  1. For the change in *a*
  2. For the change in *not_a* (after the evaluation of the continuous assignment)

- The failure during the first execution of *a_final* will be added in the DIA Report Queue
- When *not_a* changes, DIA Report Queue is flushed due to the activation of the always_comb block (flush point reached)
- No failure of *a_final* will be reported.

- Success will be added to DIA report queue and will mature and reported in the Postponed region

Assertion Usage: Recommendation

- Use final DIA rather than Simple IA or Observed DIA
  - Using keyword assert final

Recommended to use concurrent assertion rather than immediate assertion whenever possible

- Behavior of concurrent assertion easier to model and understand

Concurrent Assertion: Basics

- Supports clear specification of clock and reset
- Examine behavior over time (clock cycles)

```
check_grant: assert property(
  @(posedge clk)
  disable iff(reset)
  !(grant[0] && grant[1]))
  else $error("Two Grants simulataneously!");
```

Concurrent assertion has:

- Keywords: assert property
- Optional clock specification (with edge)
- Optional reset specification (using keywords disable iff)

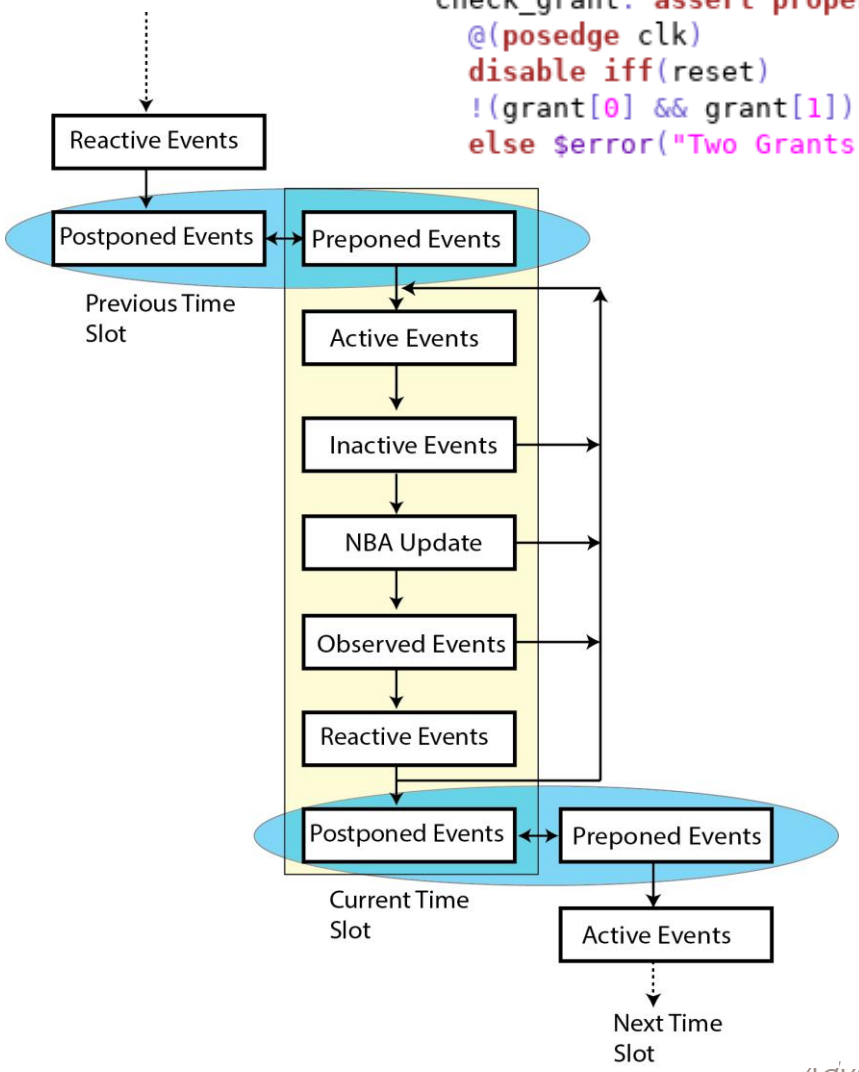
Behavior of concurrent assertions:

- Checked only at the given clock edges
- Ignored during reset (disable iff) condition

- Edge should be clearly specified for the clock: posedge or negedge
- If no edge specified (@(clk1)), checking will be done for both edges
- Clock is assumed to be glitch-free: If the clocking event transitions more than once during a time step, the resulting behavior is undefined.

Mechanism of Concurrent Assertions

```
check_grant: assert property(
  @(posedge clk)
  disable iff(reset)
  !(grant[0] && grant[1]))
  else $error("Two Grants simulataneously!");
```



Evaluation:

- Concurrent assertions are evaluated in the **Observed Region** (after design activities have settled for a time slot)
- The evaluation is done using ***sampled value*** of the signals

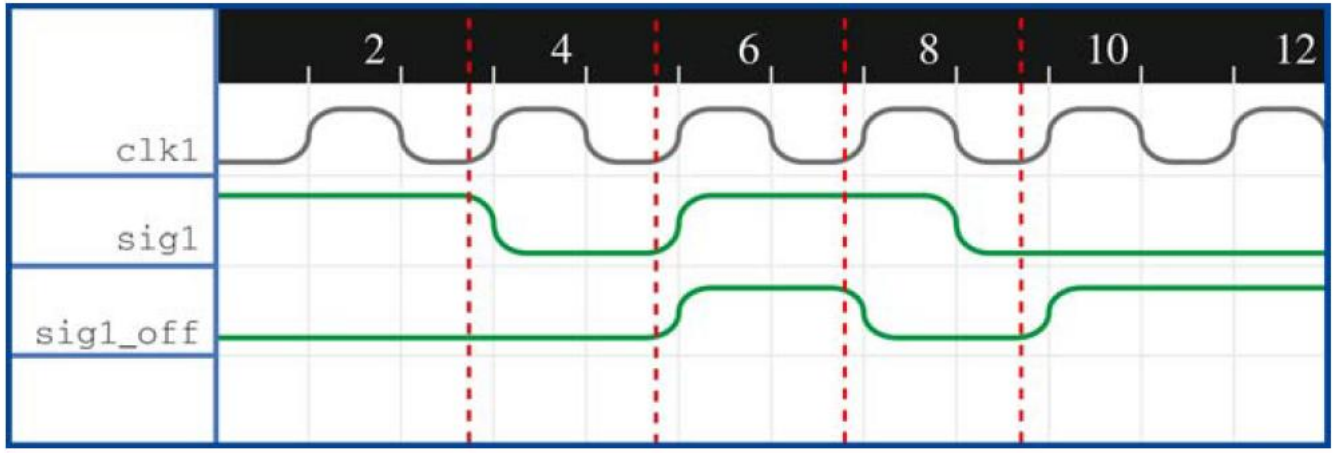
Sampled Value:

- Time slot greater than 0: value of variable in the **Preponed Region**
- Time slot equal to 0: default value of a variable
  - Example: the default value of variable of type logic is 1'bx
- If a variable is an input variable of a clocking block, the variable sampled with **#1step** sampling (just before the clock edge)

Concurrent Assertion: Sampling Example

- At a given time step, the sampled value is the value of variable just before the clock edge

```
sig1_off:
  assert property
  (@(posedge clk1) !sig1);
```



Concurrent Assertion: Sampled Value Functions

Table 3.2 The most useful sampled value functions.

Function	Description
`$past`(*expr*)	Sampled value of *expr* one clock earlier
`$past`(*expr,n*)	Like `$past`(*expr*), but look *n* clocks earlier instead of one
`$rose`(*expr*)	Returns 1 iff LSB[a] of sampled value of *expr* is 1, and one clock earlier it was non-1
`$fell`(*expr*)	Returns 1 iff LSB of sampled value of *expr* is 0, and one clock earlier it was non-0
`$stable`(*expr*)	Returns 1 iff sampled value of *expr* equals value at previous edge
`$changed`(*expr*)	Returns 1 iff sampled value of *expr* does not equal value at previous edge

Concurrent Assertion: Default Clock and Reset

- SVA allows setting a global clock and reset for concurrent assertion in a module
  - ➢ Avoid repetitive code
- Default clock specified using: default clocking statement
- Default reset specified using: default disable iff statement

```
check_grant: assert property(
  @(posedge clk)
  disable iff(reset)
  !(grant[0] && grant[1]))
  else $error("Two Grants simulataneously!");
```

```
default clocking @(posedge clk); endclocking
```

```
default disable iff(reset);
```

```
check_grant: assert property(
  !(grant[0] && grant[1]))
  else $error("Two Grants simulataneously!");
```

Concurrent Assertion: Sequences

- Properties are often constructed out of sequential behaviors.
- **SVA sequence** provides the capability to build and manipulate sequential behaviors.

- A sequence specifies a set of values occurring over a period of time

- A sequence is said to be matched if **ALL its specified values have matched**

- A sequence with a variable delay can have multiple overlapping sequences

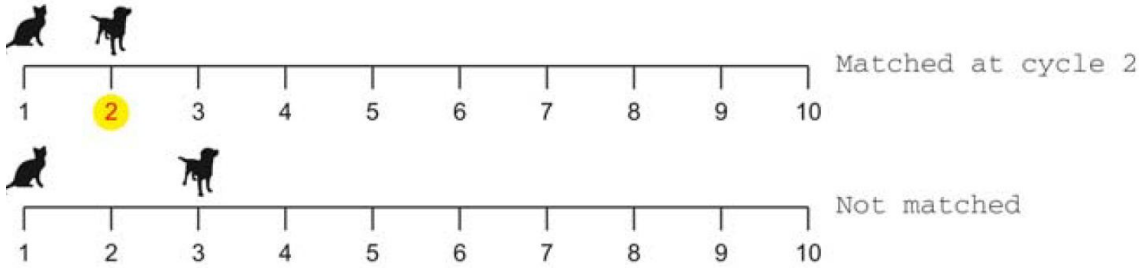
Sequences: Delay Operator

- Delay operator:
  - ##N : delay of N clock cycles
  - ##[N:M]: delay between N and M clock cycles
  - ##[N:$]: unlimited upper bound on number of cycles

Sequences: Delay Operator Examples

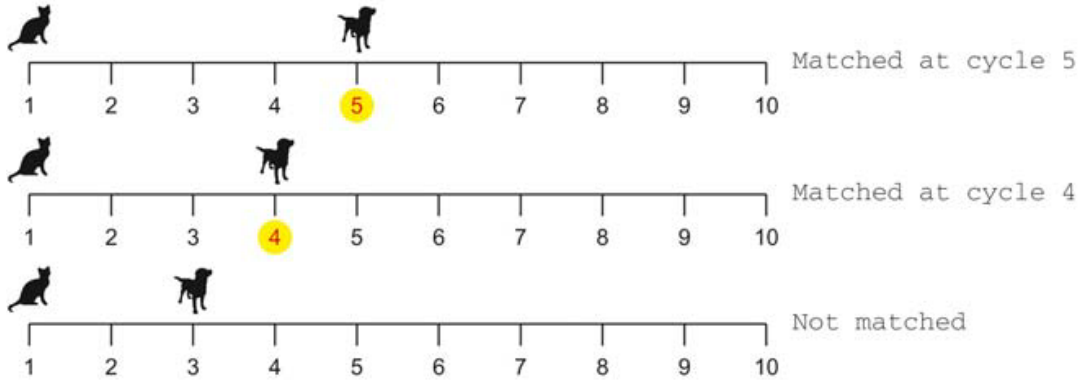
```
cat ##1 dog
```

Cat followed one cycle later by dog



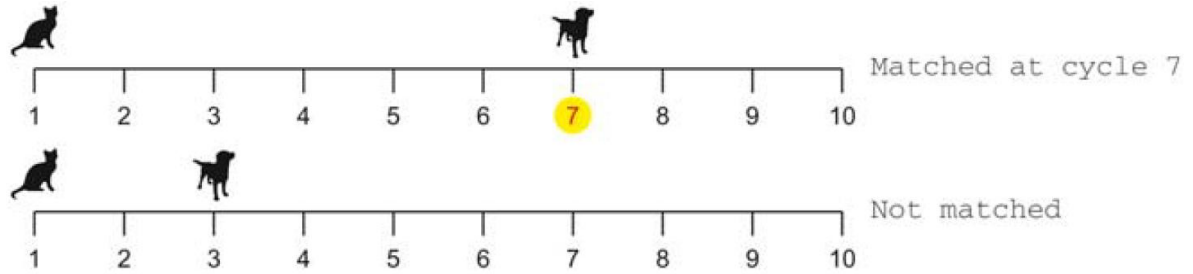

```
cat ##[3:4] dog
```

Cat followed by a dog 3 or 4 cycles later




```
cat ##[3:$] dog
```

Cat followed by a dog 3 or more cycles later



Sequences: Consecutive Repetition Operator

- [*const_or_range_expression*]
- Specifies finitely many iterative matches of the operand sequence
- **Meaning of consecutive**: a delay of one clock tick from the end of one match to the beginning of the next.
- The overall repetition sequence matches at the end of the last iterative match of the operand.

- Consecutive Repetition operator
  - [*P] : Given subsequence repeats P times
  - [*P:Q] : Given subsequence repeats P to Q times
  - [*P:$] : Given subsequence repeats P or more times
  - [*] : Equivalent to [*0:$]
  - [+] : Equivalent to [*1:$]

Sequences: Consecutive Repetition Operator Examples

```
cat [*2] ##1 dog
```

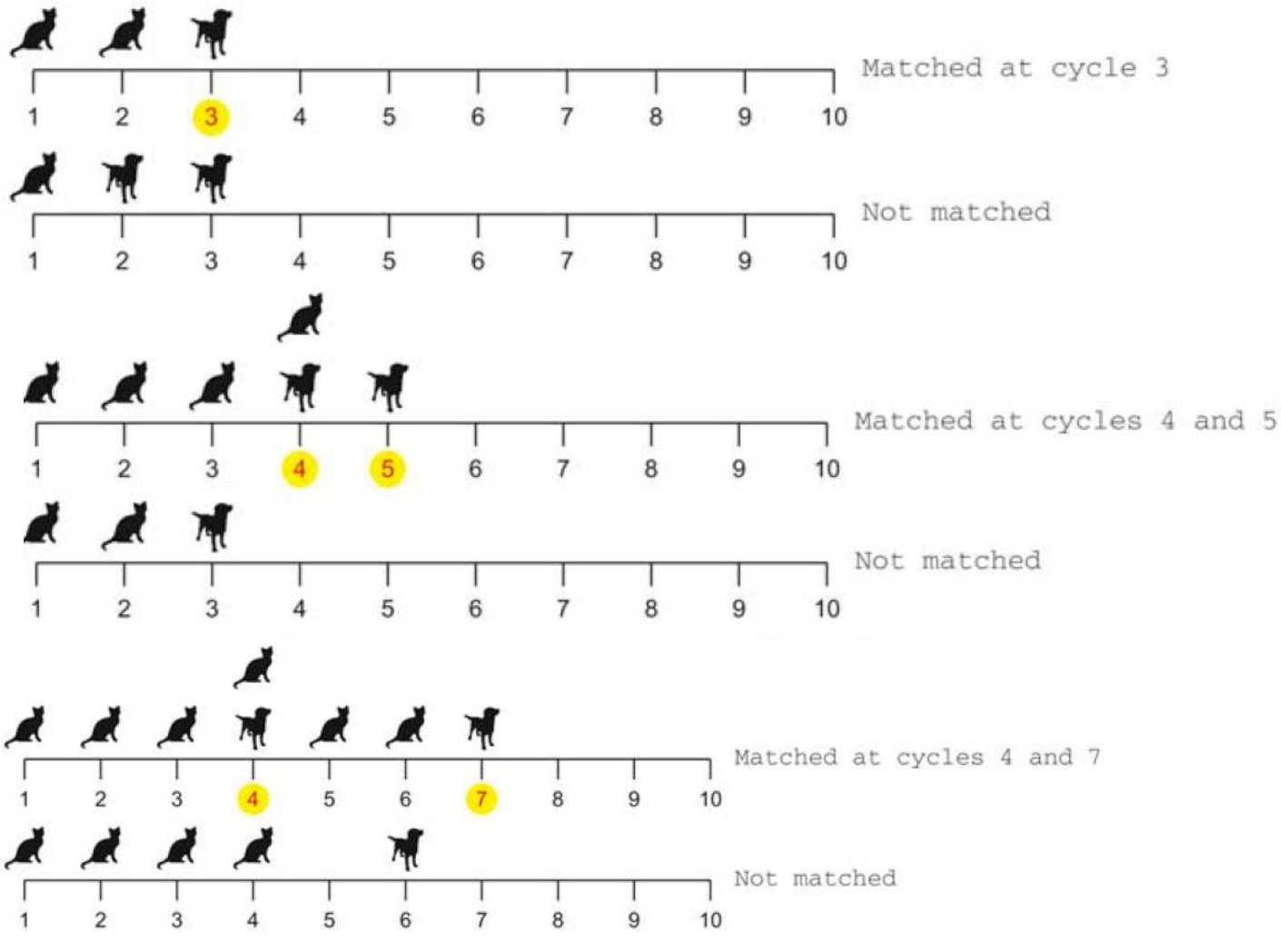
2 consecutive cats followed by a dog

```
cat [*3:4] ##1 dog
```

3 or 4 consecutive cats followed by a dog

```
cat [*3:$] ##1 dog
```

3 or more consecutive cats followed by a dog



Sequences: Other Operators

- Other operators for sequences:
  - S1 and S2: both S1 and S2 are starting at the same time
  - S1 or S2 : one of the two sequences S1 or S2 should match
  - S1 within S2: sequence S1 occurs within the execution of S2
  - E throughout S : given expression E remain true for the entire execution of sequence S

Sequences: Other Operator Examples (1)

```
cat[*2] or dog[*2]
```

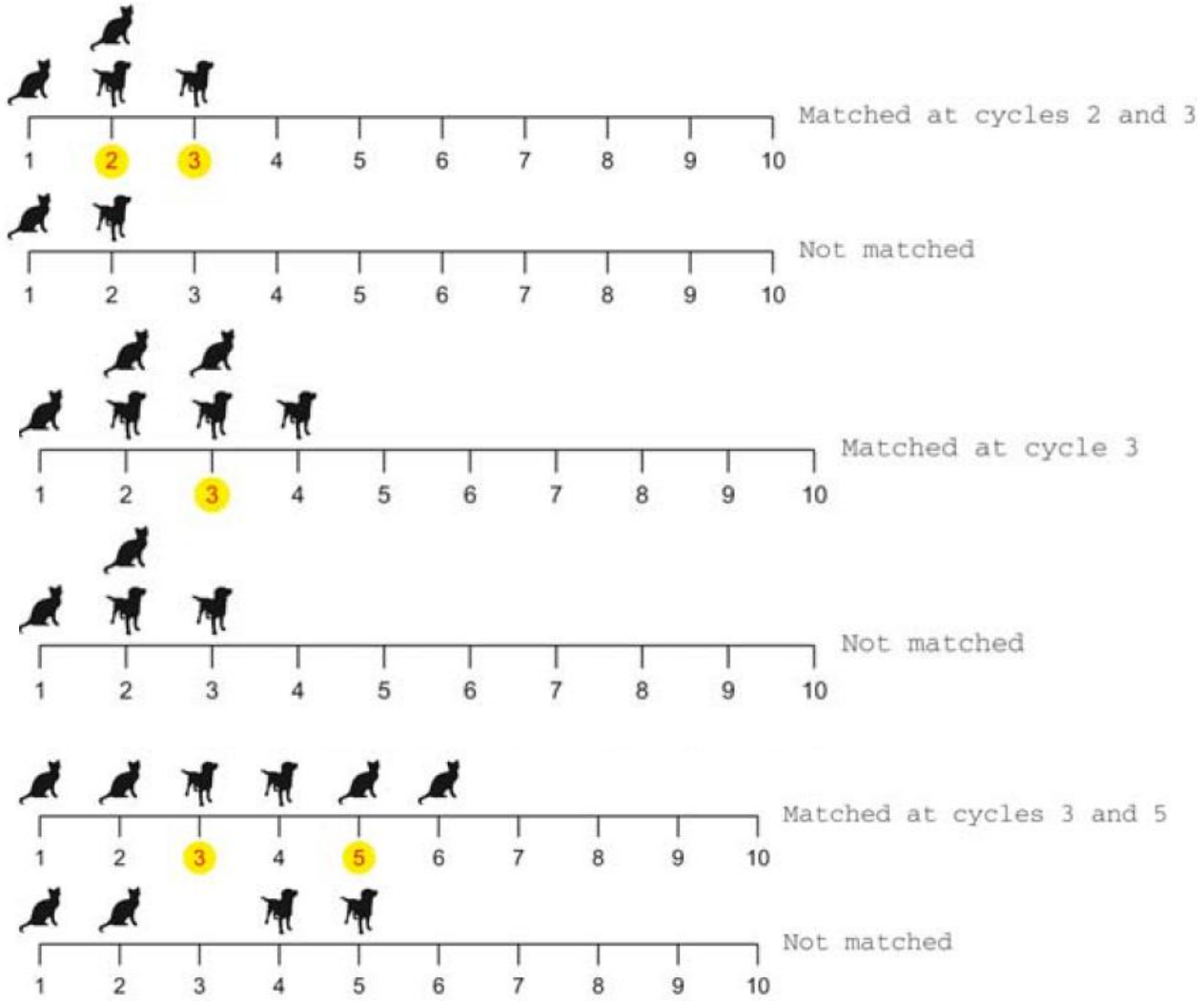
2 consecutive cats or 2 consecutive dogs

```
cat[*2] and dog[*2]
```

2 consecutive cats and 2 consecutive dogs starting at the same time

```
(cat ##1 dog) or (dog ##1 cat)
```

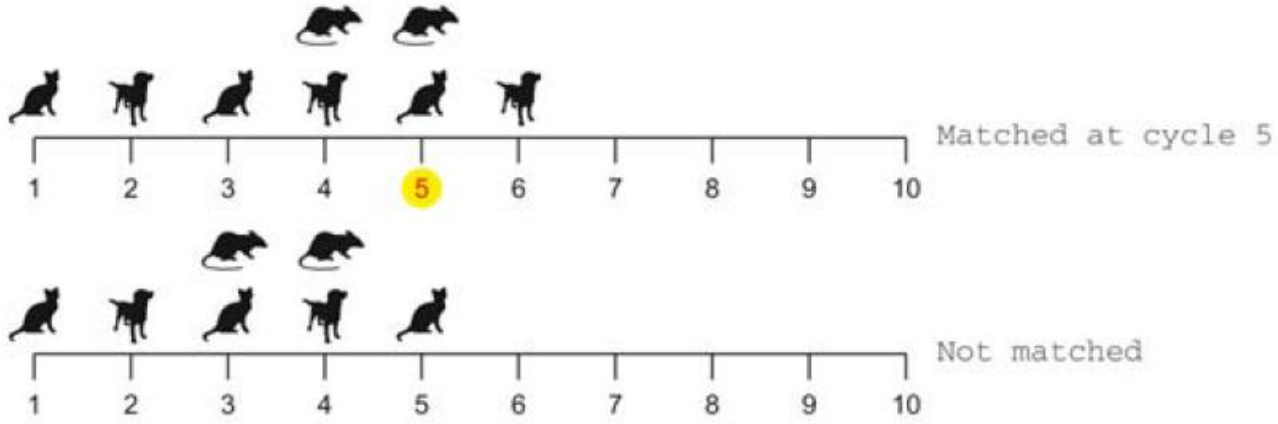
Cat followed by dog or dog followed by cat



Sequences: Other Operator Examples (2)

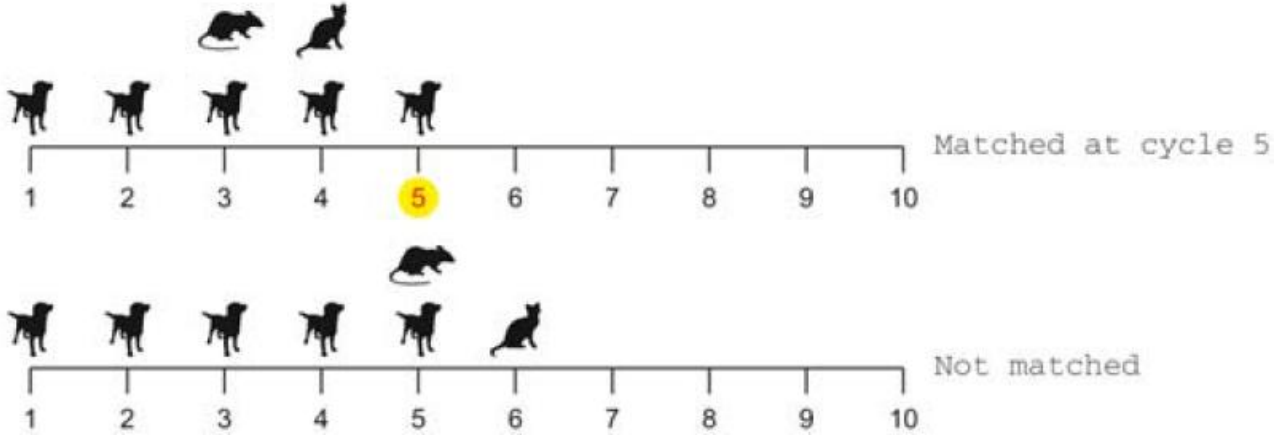
```
mouse throughout (dog ## cat)
```

Mouse should be true in the sequence in which dog is followed by cat




```
(mouse ##1 cat) within dog [*5]
```

Mouse followed by cat in the sequence of 5 consecutive dogs



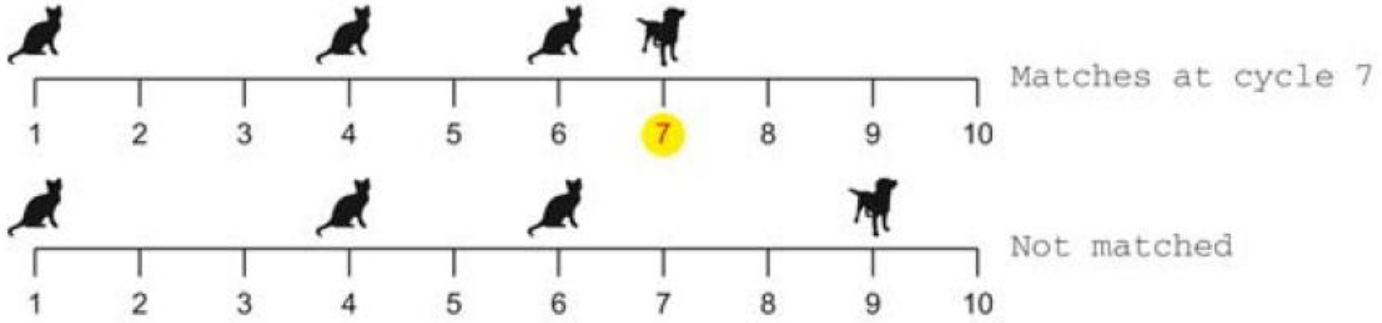
Sequences: Goto Repetition Operator

- Goto repetition operator: [->N] [=N]
  - Some value occurs N times, need not be consecutive
  - [->N]: matches at the final occurrence of the value
  - [=N]: matches at or after the final occurrence of the value

Sequences: Goto Repetition Operator Examples

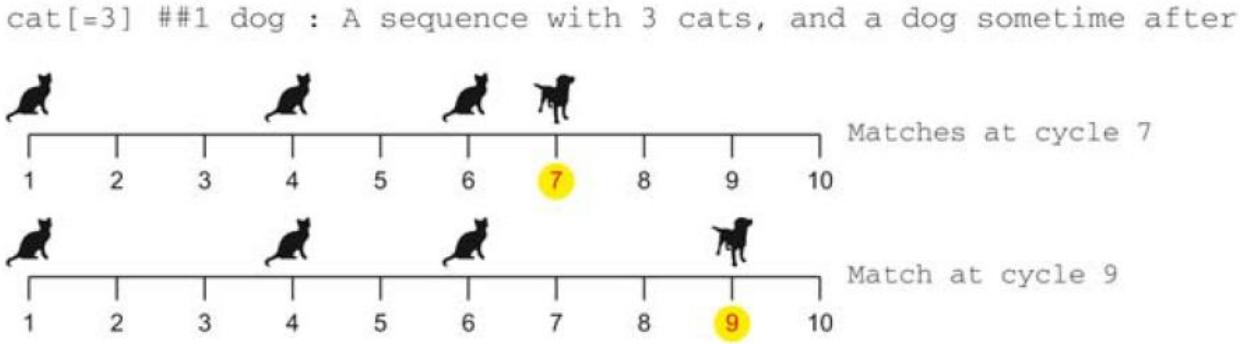
```
cat [->3] ##1 dog
```

3 cats (can be non-consecutive) followed by dog after the last cat




```
cat [=3] ##1 dog
```

3 cats (can be non-consecutive) followed by dog sometime after the last cat



Sequences: Cover Property

- Sequences are typically code fragments of assert statements
- Sequences may be useful in directly writing the cover property

```
c1: cover property (req[0] ##1 grant[0]);
```

Cover case where req[0] is true and grant[0] is true in 1 cycle

```
c2: cover property (req[0] && !grant[0][*5:$]) ##1 grant[0];
```

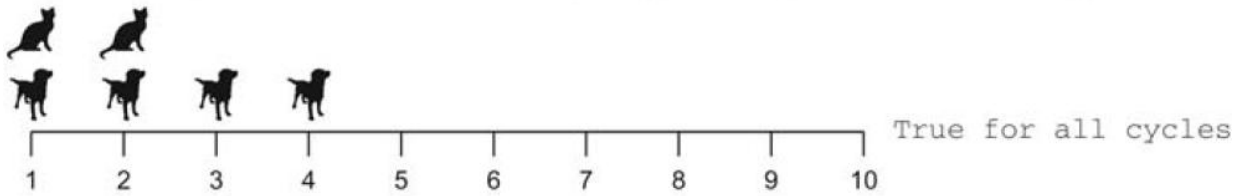
Cover case where req[0] is true and grant[0] is false for 5 or more cycle before grant[0] becoming true

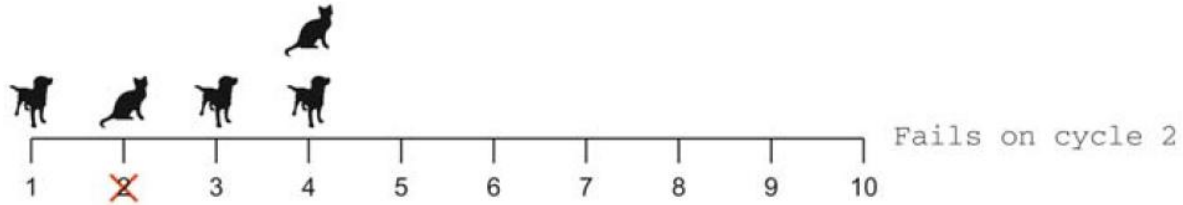
Concurrent Assertion: Implication Operators (1)

- Most common type of assertions are created using triggered implications
  - Using operators |-> or |=>

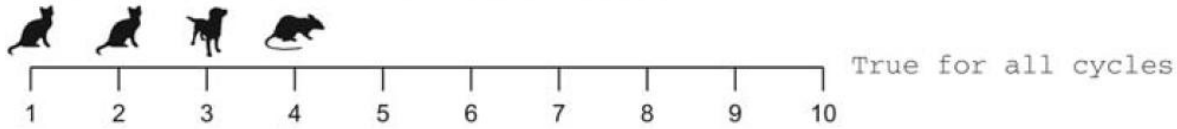
- LHS |-> RHS
- LHS |=> RHS
  - LHS is known as antecedent
  - RHS is known as consequent
- Antecedent can be a sequence
- Consequent can be a property or a sequence
- |-> checks property on the same clock tick, while |=> checks one tick later
- When antecedent is false, the property is vacuously true

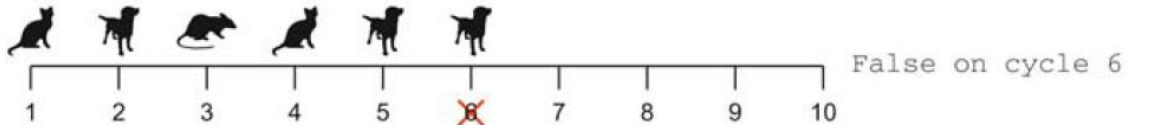
```
cat |-> dog
```





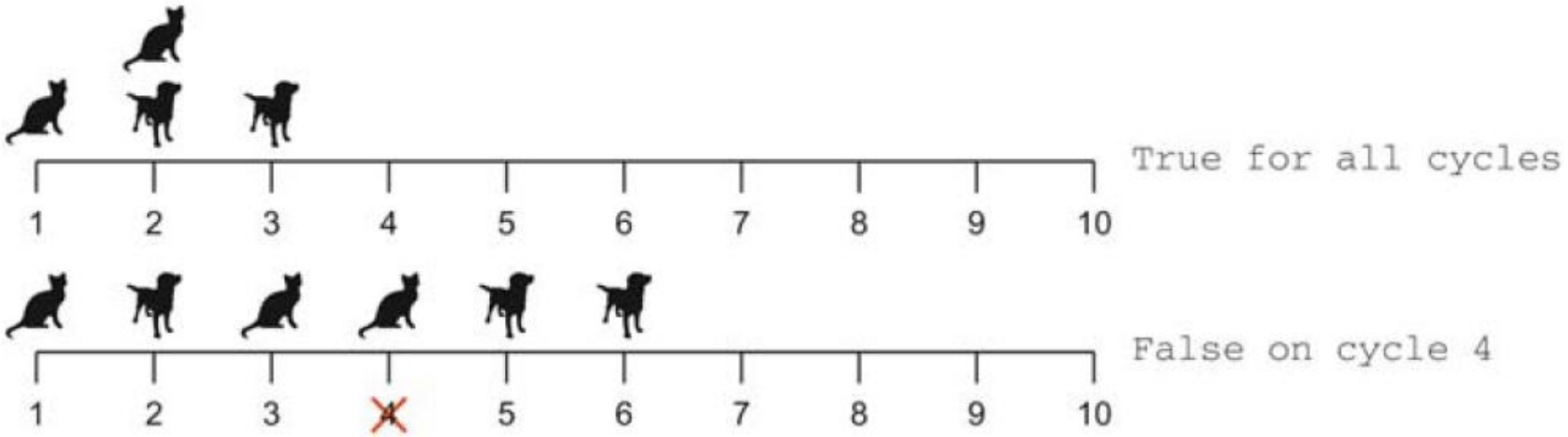

```
(cat ##1 dog) |=> mouse
```





Concurrent Assertion: Implication Operators (2)

```
(cat[*2] |-> dog[*2])
```



References

- Seligman, Erik, Tom Schubert, and MV Achutha Kiran Kumar. Formal verification: an essential toolkit for modern VLSI design. Elsevier, 2023
- Saurabh, Sneh. Introduction to VLSI design flow. Cambridge University Press, 2023.